

D213 Advanced Data Analytics
NLM3 Time Series Modeling (Task 1)
Performance Assessment

Hillary Osei (Student ID #011039266)

Western Governors University, College of Information Technology

Program Mentor: Dan Estes

August 24, 2025

Table of Contents

Part I: Research Question.....	3
Section A1. Proposal of Question.....	3
Section A2. Objectives or Goals.....	3
Part II: Method Justification.....	3
Section B. Summary of Assumptions.....	3
Part III: Data Preparation.....	4
Section C1. Line Graph Visualization.....	4
Section C2. Time Step Formatting.....	4
Section C3. Stationarity.....	4
Section C4. Steps To Prepare The Data.....	5
Section C5. Prepared Dataset.....	6
Part IV: Model Identification and Analysis.....	7
Section D1. Report Findings and Visualizations.....	7
Section D2. ARIMA Model.....	10
Section D3. Forecasting Using ARIMA Model.....	11
Section D4. Outputs and Calculations.....	14
Section D5. Code.....	23
Part V: Data Summary and Implications.....	23
Section E1. Results.....	23
Section E2. Annotated Visualization.....	1
Section E3. Recommendation.....	1
Part VI: Reporting.....	1
Section F. Reporting.....	1
Section G. Sources for Third-Party Code.....	1
Section H. Sources.....	1

Part I: Research Question

Section A1. Proposal of Question

The research question addressed is “Can we forecast daily revenue in the medical industry over the remainder of the year using a time-series model?”

Section A2. Objectives or Goals

The goal of this analysis is to accurately forecast daily revenue based on the provided dataset “medical_time_series.csv.” 80% of the data will be used as a training set and the remaining 20% will be used as a test set for evaluation. This allows the forecast to be compared against actual values and therefore the model performance can also be measured. An ARIMA time-series model is applied to the training data to project revenue patterns and assess how well the model predicts.

Part II: Method Justification

Section B. Summary of Assumptions

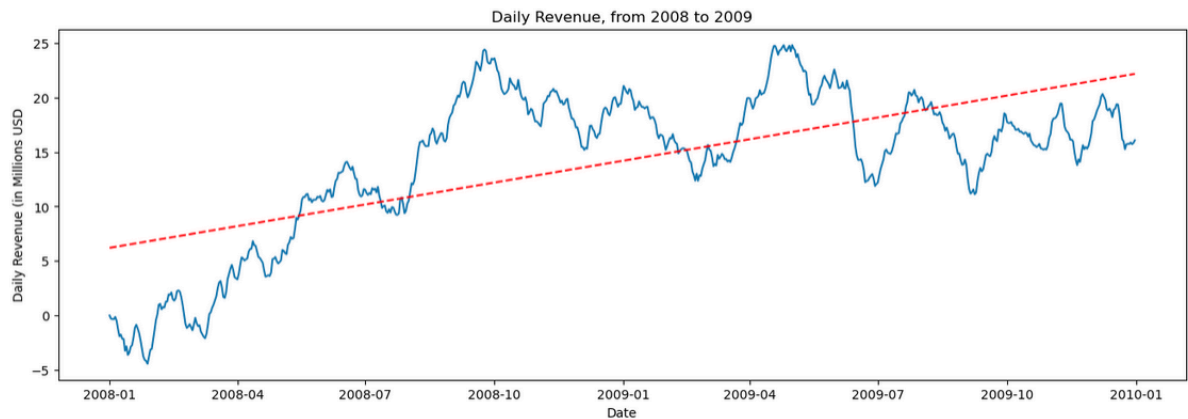
Time-series models have several assumptions. One assumption is that the time series model must show stationarity. (Kuo, 2024) Stationarity means that it has a constant mean, variance, and autocorrelation structure over time (NIST, n.d.). Additionally, the model assumes that errors are uncorrelated, the data has no outliers, and that any random shocks, or unexpected events or disruptions in the data, are completely random and have the mean of 0 and constant variance (Kuo, 2024).

Part III: Data Preparation

Section C1. Line Graph Visualization

Below is the visualization of the line graph showing daily revenue from 2008-2009.

```
[5]: # Plot time series model
plt.figure(figsize = [16,5])
plt.title("Daily Revenue, from 2008 to 2009")
plt.xlabel("Date")
plt.ylabel("Daily Revenue (in Millions USD)")
plt.plot(df)
# Generate trend line
x = mdates.date2num(df.index)
y = df.Revenue
z = np.polyfit(x, y, 1)
p = np.poly1d(z)
# Plot trendline
plt.plot(x, p(x), "r--")
plt.show()
```



Section C2. Time Step Formatting

The dataset is structured with a DateTime index and daily revenue observations measured in millions of U.S. dollars/ The index increased in one-day increments from January 1, 2008 to December 231, 2009. resulting in a total of 731 records. The count also takes into account the leap year in 2008, adding an extra day in February. No gaps or missing values are found in the series. By formatting the dates as DateTime object, the data can be easily aggregated for further analysis, for example monthly and yearly revenue.

Section C3. Stationarity

The data shows that the data is non-stationarity. With stationarity, the mean and variance remain constant over time, however the graph shows a clear upward trend. The graph shows that daily revenues neared 0 but by the tail of the graph, daily revenue grew to 15-20 million dollars. This upward trend is also evident in the trend line included, which slopes upward. To statistically

confirm this, an Augmented Dickey-Fuller test was confused. The results produced a test statistic of -2.2183 and a p-value of 0.1997. The p-value is greater than 0.05, therefore confirming that the data is non-stationary (Prabhakaran, 2019).

```
[6]: # Perform Augmented Dickey-Fuller
adfuller_results = adfuller(df.Revenue)
# Print resulting test-statistic and p-value
print(f"The test statistic of Dickey-Fuller test on this data is {round(adfuller_results[0], 4)}, which has a p-value of {round(adfuller_resul

The test statistic of Dickey-Fuller test on this data is -2.2183, which has a p-value of 0.1997
```

Section C4. Steps To Prepare The Data

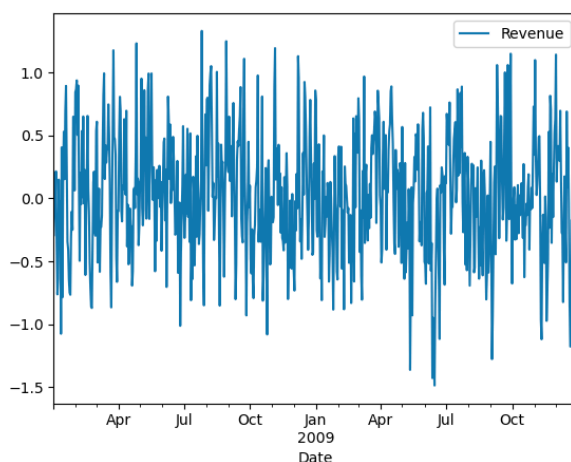
The preparation of the dataset began by importing the file “medical_time_series.csv” and converting the raw day values into a proper Datetime index. This was done to ensure that each value aligned with a calendar date. After the index was set, the data was visualized in a line plot, which showed that there is an upward trend and that the series was not stationary. To test for stationarity, the Augmented Dickey-Fuller test was applied to the original series. The results showed a test statistic of -2.2183 and a p-value of 0.1997, meaning the original data was non-stationary. To address this issue, the difference in the dataset and all null values were dropped. The ADF test was performed again on the differenced data, resulting in a test statistic of -17.3748 and a p-value of 0.0. Because the p-value is less than 0.05, this confirms that the differenced data is stationary and therefore appropriate for ARIMA modeling. When the series was transformed, it was divided into a training set and testing set, with 80% being used for training and 20% being used for testing. Then, both training and testing sets were saved into separate CSV files.

```
[7]: # Take the difference of the dataset
df_trans = df.diff().dropna()
# Perform Augmented Dickey-Fuller on data
adfuller_results = adfuller(df_trans['Revenue'])
# Print resulting test-statistic and p-value
print(f"The test statistic of an augmented Dickey-Fuller test on this data is {round(adfuller_results[0], 4)}, which has a p-value of {round(a

# Plot to verify data is stationary
df_trans.plot()

The test statistic of an augmented Dickey-Fuller test on this data is -17.3748, which has a p-value of 0.0

[7]: <Axes: xlabel='Date'>
```



```
[8]: # Split time series into a training set and a test set
train, test = train_test_split(df_trans, test_size=0.2, shuffle=False, random_state=369)
train
```

```
[8]:
```

Revenue	
Date	
2008-01-02	-0.292356
2008-01-03	-0.035416
2008-01-04	-0.012215
2008-01-05	0.215100
2008-01-06	-0.366702
...	...
2009-08-03	-0.694370
2009-08-04	-0.282765
2009-08-05	0.104732
2009-08-06	0.275857
2009-08-07	0.126645

584 rows × 1 columns

```
[9]: test
```

```
[9]:
```

Revenue	
Date	
2009-08-08	0.263991
2009-08-09	-0.588690
2009-08-10	-0.550427
2009-08-11	0.081477
2009-08-12	-0.146587
...	...
2009-12-27	-0.032693
2009-12-28	0.143766
2009-12-29	-0.156834
2009-12-30	0.113880
2009-12-31	0.246562

146 rows × 1 columns

Section C5. Prepared Dataset

Datasets are attached, titled “d213_task1_train_clean.csv” and “d213_task1_test_clean.csv”

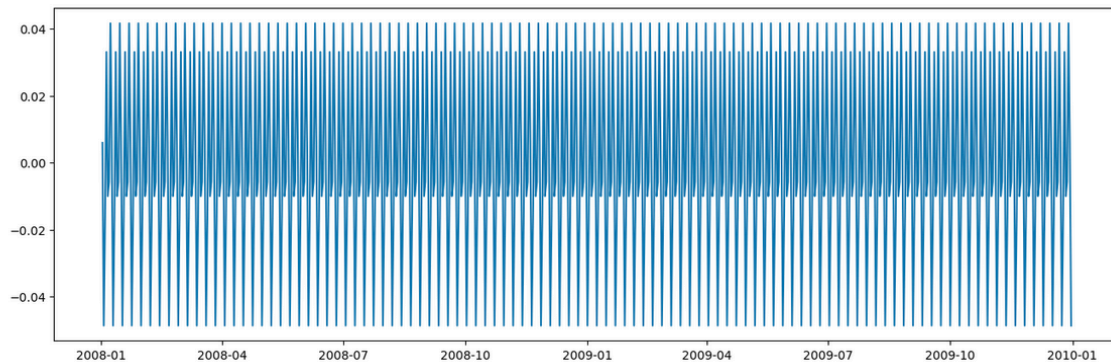
Part IV: Model Identification and Analysis

Section D1. Report Findings and Visualizations

The seasonal decomposition shows a consistent pattern across the dataset.

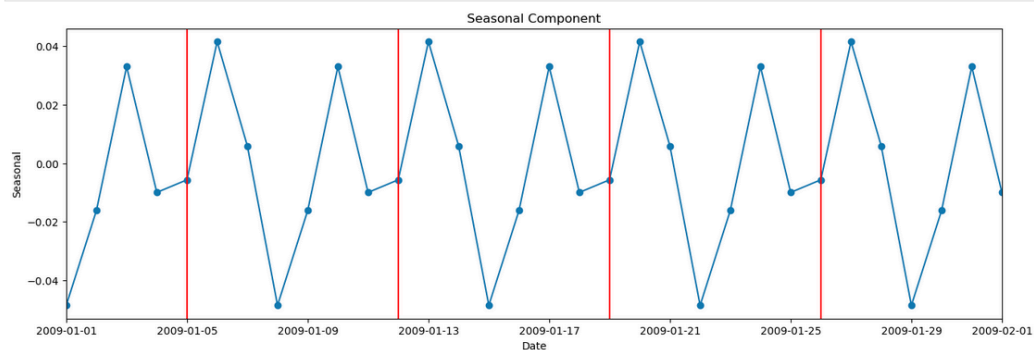
```
[11]: # Decompose the transformed data
decomposed_data = seasonal_decompose(df_trans)
# Long X and small Y dictate a wide graph figure
plt.figure(figsize = [16,5])
# Plot seasonal component of the data
plt.plot(decomposed_data.seasonal)
```

```
[11]: [<matplotlib.lines.Line2D at 0x129fc49d0>]
```



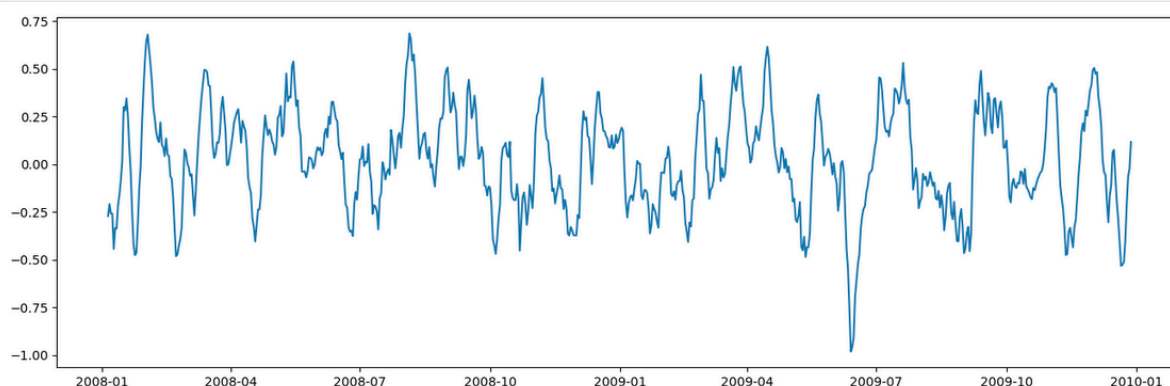
The seasonal component oscillated about 0.04 units around zero, while the broader datasets varied from 1.0 to 1.5. On closer inspection from January 2009 to February 2009 shows that the seasonality aligns with the days of the week. Using Mondays as reference points, the plot shows that there are two peaks occurring each week, usually on Saturdays and Tuesdays while there is a noticeable drop on Sundays and Thursdays.

```
[12]: # Plot seasonality
plt.figure(figsize = [16,5])
# Plot seasonal component of the data
plt.plot(decomposed_data.seasonal, marker='o')
plt.xlim(pd.to_datetime('2009-01-01'), pd.to_datetime('2009-02-01'))
# Draw red lines on Mondays
plt.axvline(x=pd.to_datetime('2009-01-05'), color='red')
plt.axvline(x=pd.to_datetime('2009-01-12'), color='red')
plt.axvline(x=pd.to_datetime('2009-01-19'), color='red')
plt.axvline(x=pd.to_datetime('2009-01-26'), color='red');
# Plot info
plt.title('Seasonal Component')
plt.xlabel('Date')
plt.ylabel('Seasonal')
plt.show()
```



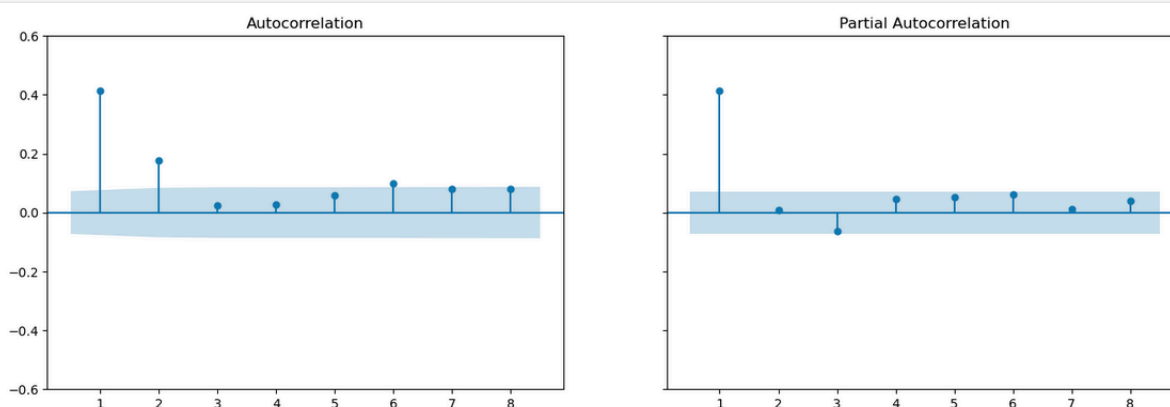
The trend component for the decomposed data doesn't show any upward or downward movement, confirming that the trend has been completely removed. However, the plot does show that there is an outlier somewhere in June of 2009.

```
[13]: # Plot trends
plt.figure(figsize = [16,5])
plt.plot(decomposed_data.trend);
```



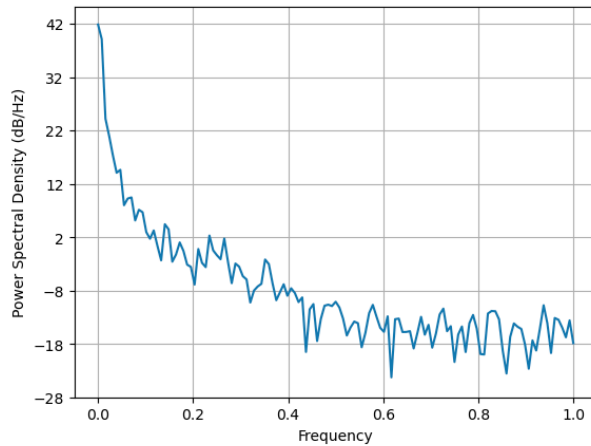
For the autocorrelation and partial autocorrelation plots, the ACF shows a gradual decline, tailing off after lag 2 and the PACF shows a sharp cutoff at lag 1. According to Datacamp, an ACF that tails off and a PACF that has a cutoff at lag p indicates an AR model of order p (Fulton, n.d.). This means that the data is suited for an AR(1) model.

```
[14]: # Plot autocorrelation and partial autocorrelation
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=[16,5], sharey=True)
# Plot ACF to up to 8 lags to represent days
plot_acf(df_trans, lags=8, zero=False, ax=ax1)
# Plot PACF to 8 up to 8 lags to represent days
plot_pacf(df_trans, lags=8, zero=False, ax=ax2)
# Zoom in on y-axis to see points clearly
plt.ylim(-0.6, 0.6)
plt.show()
```



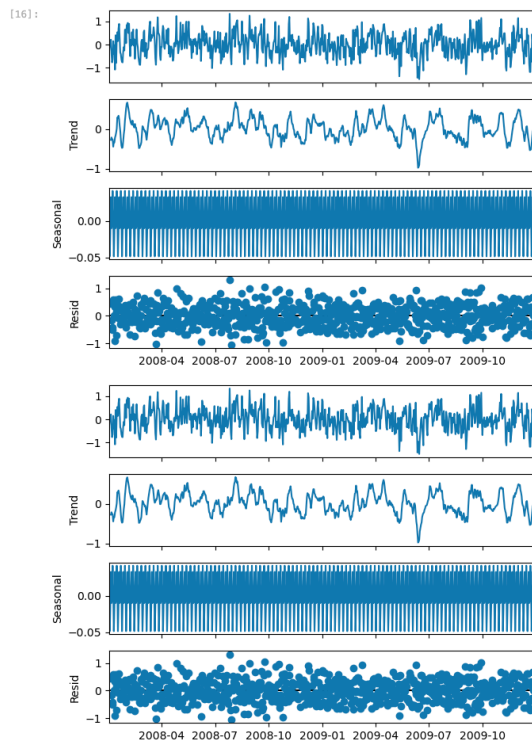
Below is the spectral density plot, or power spectral density plot, which measures how the strength of a signal is distributed across different frequencies (Jain, 2024).

```
[15]: # Plot spectral density
plt.psd(df['Revenue'])
plt.show()
```



Below are the decomposed time series plots, which separates a time series into several components: trend, seasonality, and residual/noise (Jain, 2023).

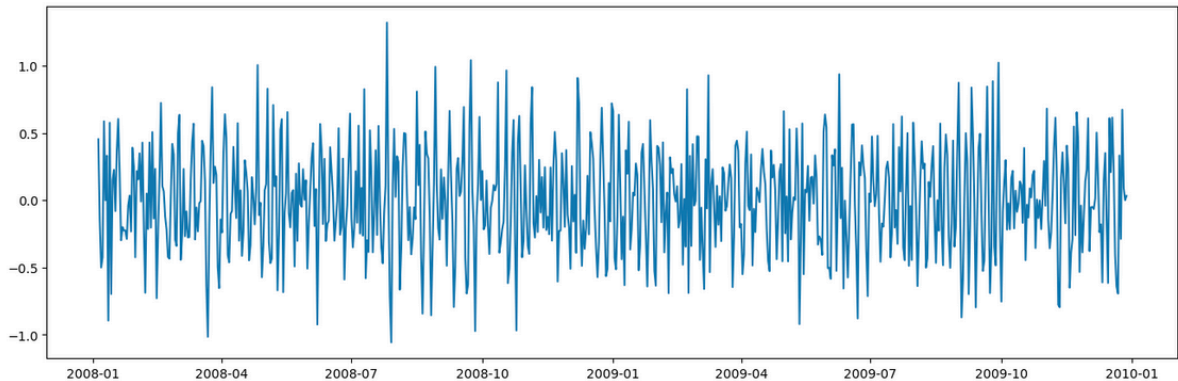
```
[16]: # Plot decomposed data
decomposed_data.plot()
```



Below is a plot confirming that there is no trend present in the residuals of decomposition. This is evident in the multiple, random pikes and low datapoints in the plot.

```
[17]: # Show lack of trends in residuals of decomposition
plt.figure(figsize = [16,5])
# Plot residual component of the data
plt.plot(decomposed_data.resid)
```

```
[17]: [matplotlib.lines.Line2D at 0x12a2ff610>]
```



Section D2. ARIMA Model

An ARIMA(1,0,0) model was fitted to the training data, which aligns with the results from the ACF and PACF plots. The ACF showed a gradual tailing off after lag 2 while PACF showed a sharp cutoff after lag 1. This pattern, an ACF that tails off and a PACF that cuts off after lag p , is an indication that the series should be modeled with an autoregressive component of order p (Fulton, n.d.). In this case, the cutoff at lag 1 in the PACF justifies selecting an AR(1) model.

The results of the fitted model shows that the AR(1) coefficient is at 0.4079 with a p-value of 0.000, showing a strong statistical significance since it is less than 0.05. This also means that if revenue goes up by 1 unit, the next day's revenue will also increase by 0.4079 units. The constant coefficient is at 0.0328 and has a p-value of 0.288 indicating that there is no statistical

significance.

```
[18]: # Perform ARIMA model
model = ARIMA(train, order=(1, 0, 0), freq='D')
results = model.fit()
print(results.summary())
```

SARIMAX Results

Dep. Variable:	Revenue	No. Observations:	584
Model:	ARIMA(1, 0, 0)	Log Likelihood	-350.349
Date:	Wed, 20 Aug 2025	AIC	706.698
Time:	10:37:15	BIC	719.808
Sample:	01-02-2008	HQIC	711.808
	- 08-07-2009		
Covariance Type:	opg		

	coef	std err	z	P> z	[0.025	0.975]
const	0.0328	0.031	1.063	0.288	-0.028	0.093
ar.L1	0.4079	0.038	10.748	0.000	0.333	0.482
sigma2	0.1943	0.012	15.948	0.000	0.170	0.218

Ljung-Box (L1) (Q):	0.10	Jarque-Bera (JB):	1.80
Prob(Q):	0.75	Prob(JB):	0.41
Heteroskedasticity (H):	1.04	Skew:	-0.05
Prob(H) (two-sided):	0.78	Kurtosis:	2.75

Warnings:

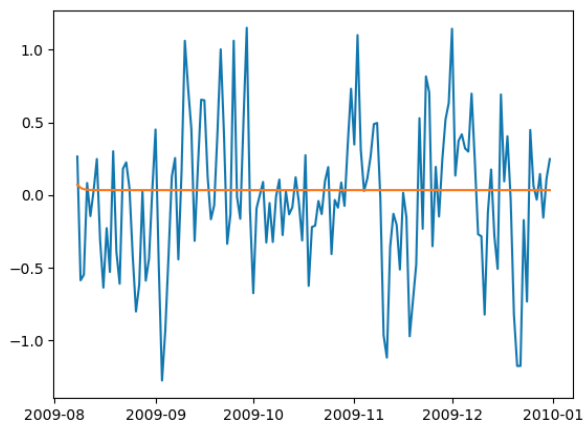
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Section D3. Forecasting Using ARIMA Model

To evaluate the performance of the ARIMA(1,0,0) model, forecasts were generated for the test period from August to December 2009. The forecast line in orange stays mostly flat near zero, while the blue line representing datapoints reveals severe daily fluctuations.

```
[19]: # Perform forecast on ARIMA model
forecasted = results.get_prediction(start = 584, end = 729, dynamic = True)
plt.plot(test)
plt.plot(forecasted.predicted_mean)
```

[19]: [<matplotlib.lines.Line2D at 0x12a0e1e90>]



The forecasted mean values represent daily differences in revenue rather than revenue levels, The mean value started at 0.07 on August 8, 2009, but as time progressed, the predicted mean values settled to 0.032791 and remained there for the rest of the year, until December 31, 2009. The values below show a predicted daily change in \$0.032791 million.

```
[20]: # Print forecasted predicted mean
print(forecasted.predicted_mean)

2009-08-08    0.071071
2009-08-09    0.048405
2009-08-10    0.039159
2009-08-11    0.035388
2009-08-12    0.033850
...
2009-12-27    0.032791
2009-12-28    0.032791
2009-12-29    0.032791
2009-12-30    0.032791
2009-12-31    0.032791
Freq: D, Name: predicted_mean, Length: 146, dtype: float64
```

To bring the forecasts back to the original scale, the predicted differences were inverted using cumulative sum and combined with the training data. The forecasted revenues started with a dip at -0.2923256. The revenue started fluctuating, decreasing and increasing rapidly. However, at the end of the year, revenue reached \$24.16 million.

```
[21]: # Create dataframe to hold forecasted differences values
forecast_temp = pd.DataFrame(forecasted.predicted_mean)
# Rename columns
forecast_temp.rename(columns={'predicted_mean': 'Revenue'}, inplace=True)
# Concat a copy of train and forecasted values
forecast_df = pd.concat([train.copy(), forecast_temp.copy()])
# Invert the differences in daily revenue using cumsum
forecast_df = forecast_df.cumsum()
# Print forecast_df
forecast_df
```

```
[21]:
```

	Revenue
2008-01-02	-0.292356
2008-01-03	-0.327772
2008-01-04	-0.339987
2008-01-05	-0.124888
2008-01-06	-0.491590
...	...
2009-12-27	24.033683
2009-12-28	24.066474
2009-12-29	24.099265
2009-12-30	24.132056
2009-12-31	24.164846

730 rows x 1 columns

Confidence intervals were also calculated for the forecasts. At the beginning of the forecast in August, the intervals were somewhat narrow (between \$18.5 million and \$20.2 million). However, as time progressed, the intervals widened greatly, from less than -\$110 million to more

than \$160 million.

```
[22]: # Calculate confidence intervals from forecasted data
confidence_intervals = forecasted.conf_int()
# Print confidence intervals
confidence_intervals
```

```
[22]:
```

	lower Revenue	upper Revenue
2009-08-08	-0.792856	0.934998
2009-08-09	-0.884621	0.981430
2009-08-10	-0.904871	0.983190
2009-08-11	-0.910461	0.981237
2009-08-12	-0.912301	0.980001
...	...	...
2009-12-27	-0.913421	0.979002
2009-12-28	-0.913421	0.979002
2009-12-29	-0.913421	0.979002
2009-12-30	-0.913421	0.979002
2009-12-31	-0.913421	0.979002

146 rows x 2 columns

```
[23]: # Create dataframe to match the confidence intervals dataframe
previous_row = pd.DataFrame({'lower Revenue': [19.312734], 'upper Revenue': [19.312734], 'Date': ['2009-08-07']})
# Convert date string to datetime and then set as index
previous_row['Date'] = pd.to_datetime(previous_row['Date'])
previous_row.set_index('Date', inplace=True)
# Print
previous_row
```

```
[23]:
```

	lower Revenue	upper Revenue
Date		
2009-08-07	19.312734	19.312734

```
[24]: # Concatenate the prior row and the confidence intervals data
confidence_intervals = pd.concat([previous_row, confidence_intervals])
# Untransform the confidence intervals using cumsum()
confidence_intervals = confidence_intervals.cumsum()
# Omit first row
confidence_intervals = confidence_intervals.loc['2009-08-08' : '2009-12-31']
# Print confidence intervals
confidence_intervals
```

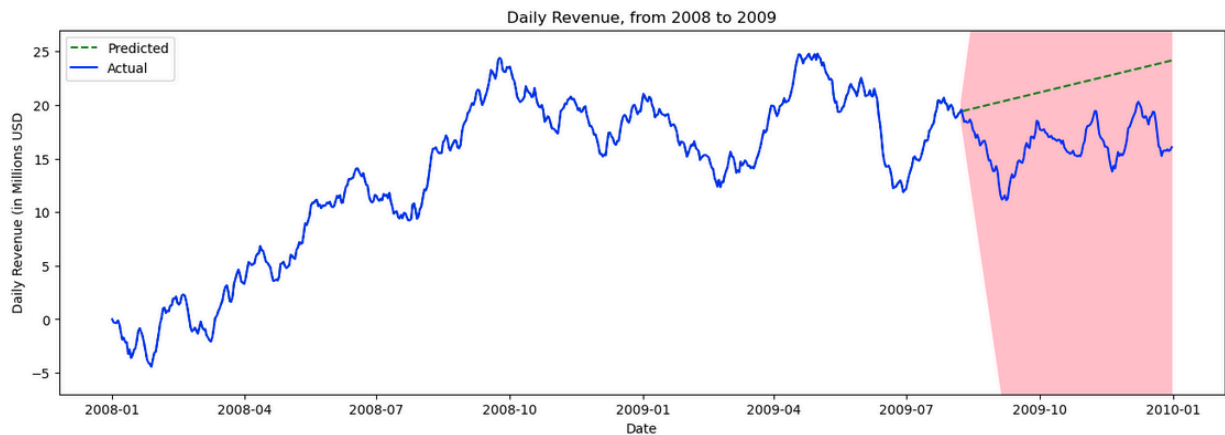
```
[24]:
```

	lower Revenue	upper Revenue
2009-08-08	18.519878	20.247732
2009-08-09	17.635257	21.229163
2009-08-10	16.730386	22.212353
2009-08-11	15.819925	23.193590
2009-08-12	14.907624	24.173592
...	...	...
2009-12-27	-110.230261	158.297627
2009-12-28	-111.143681	159.276629
2009-12-29	-112.057102	160.255632
2009-12-30	-112.970522	161.234634
2009-12-31	-113.883943	162.213636

146 rows x 2 columns

Below, the plot compares observed daily revenues (blue) with the ARIMA(1,0,0) forecast (dashed green) and the confidence interval (pink). While the forecast predicts an upward direction in revenues, it does not align with the actual, observed data. The observed revenues in late 2009 show declines and fluctuations, while the forecast has a steadily rising line. This gap shows that

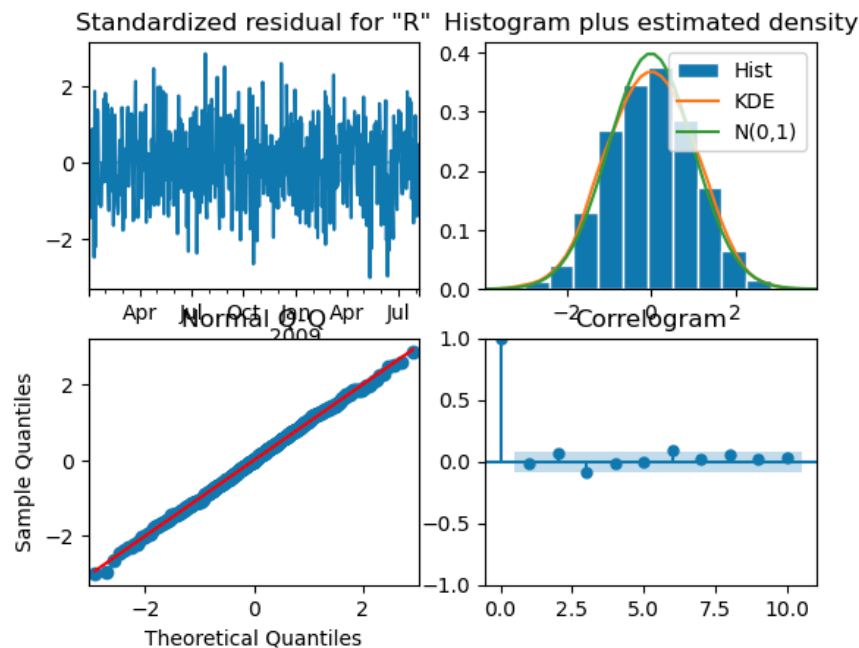
the forecast overestimates revenues compared to the actual values.



Below are diagnostic plots for the model's performance. On the top left, the standardized residual plot shows no clear pattern, indicating that the residuals are primarily random. On the top right, the histogram and kernel density estimate graph has a normal distribution, meaning that the residuals are distributed normally. The Q-Q plot on the bottom left shows that most of the points fall directly on the line, confirming normality. On the bottom right, the correlogram shows that most of the autocorrelations fall within the shaded area, however there are a few lags around 3 and 6 that are very close to being outside the area.

```
plt.figure(figsize = [16,16])
results.plot_diagnostics()
```

<Figure size 1600x1600 with 0 Axes>



The forecasting accuracy was measured using root mean squared error (RMSE), which was found to be 5.7584.

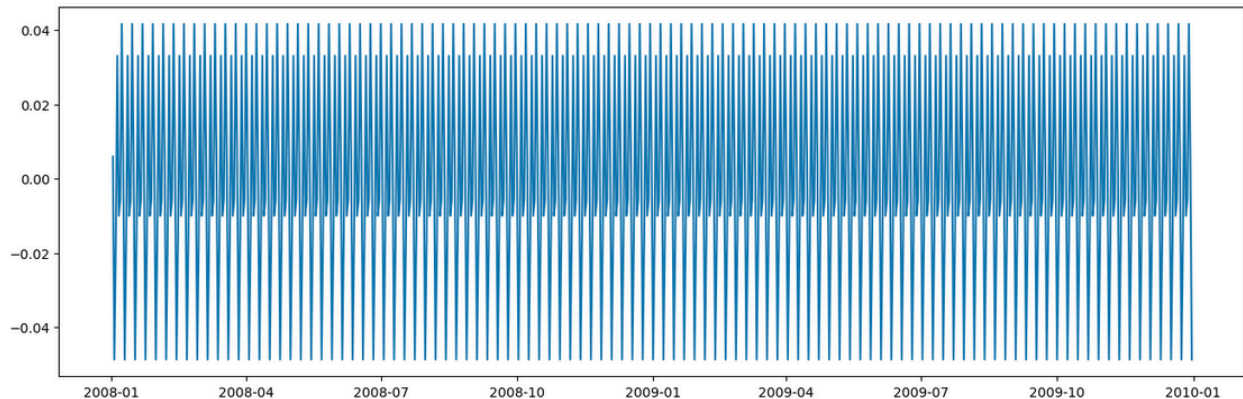
```
# Calculate root mean squared error of forecasted data vs the observed data
rmse = root_mean_squared_error(df.loc['2009-08-08' : '2009-12-31'], forecast_df.Revenue.loc['2009-08-08' : '2009-12-31'])
print(f"The root mean squared error of this forecasting model is {round(rmse, 5)}")
```

The root mean squared error of this forecasting model is 5.7584

Section D4. Outputs and Calculations

Seasonal component:

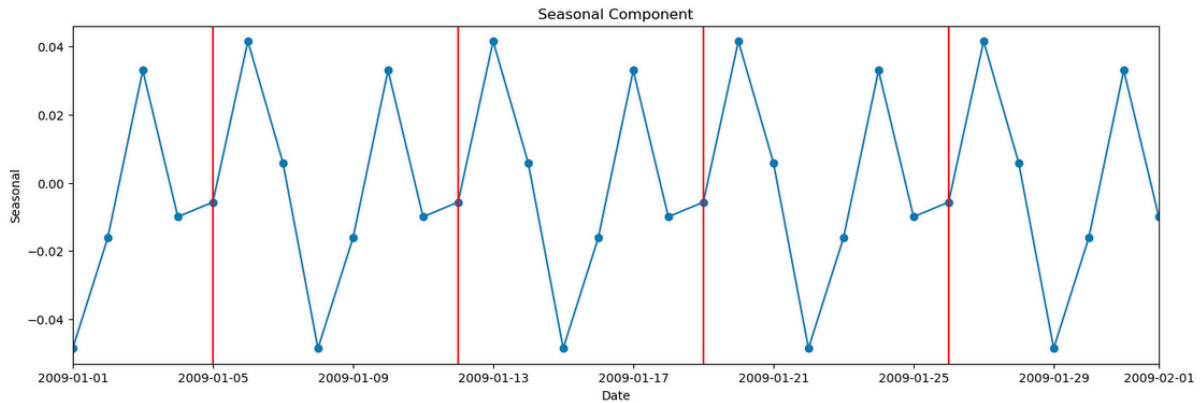
```
# Decompose the transformed data
decomposed_data = seasonal_decompose(df_trans)
# Long X and small Y dictate a wide graph figure
plt.figure(figsize = [16,5])
# Plot seasonal component of the data
plt.plot(decomposed_data.seasonal)
```



Seasonality:

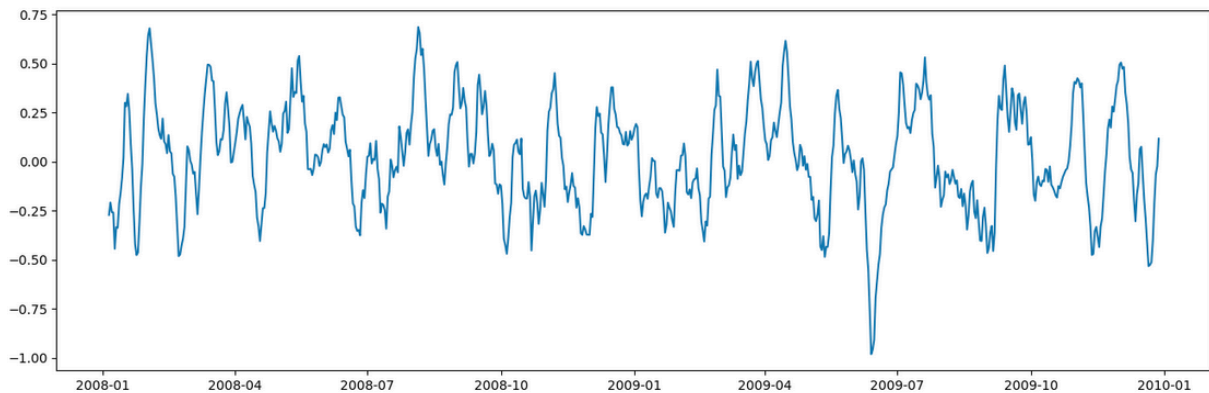
```
# Plot seasonality
plt.figure(figsize = [16,5])
# Plot seasonal component of the data
plt.plot(decomposed_data.seasonal, marker='o')
plt.xlim(pd.to_datetime('2009-01-01'), pd.to_datetime('2009-02-01'))
# Draw red lines on Mondays
plt.axvline(x=pd.to_datetime('2009-01-05'), color='red')
plt.axvline(x=pd.to_datetime('2009-01-12'), color='red')
plt.axvline(x=pd.to_datetime('2009-01-19'), color='red')
plt.axvline(x=pd.to_datetime('2009-01-26'), color='red');
# Plot info
plt.title('Seasonal Component')
```

```
plt.xlabel('Date')
plt.ylabel('Seasonal')
plt.show()
```



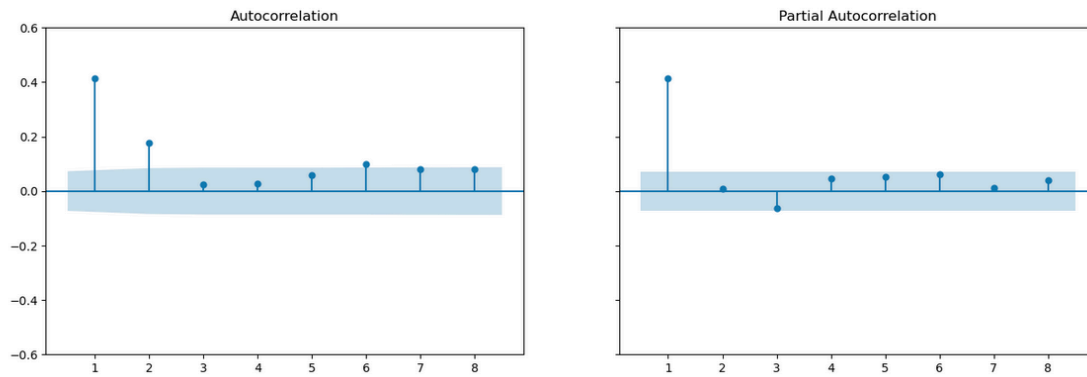
Trends:

```
# Plot trends
plt.figure(figsize = [16,5])
plt.plot(decomposed_data.trend);
```



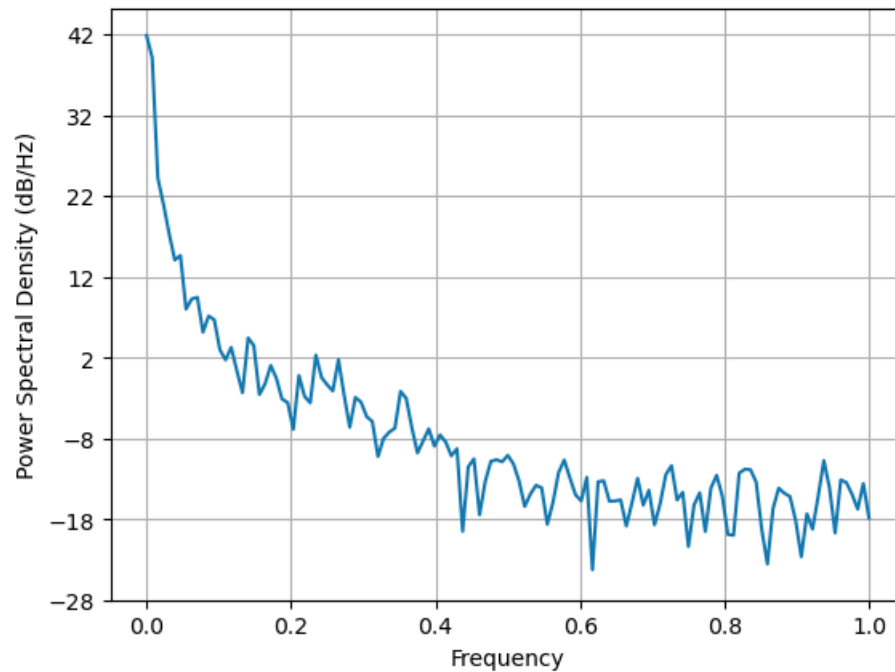
Autocorrelation & Partial autocorrelation:

```
# Plot autocorrelation and partial autocorrelation
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=[16,5], sharey=True)
# Plot ACF to 8 lags (because only 7 days in a week) & ignore zero
plot_acf(df_trans, lags=8, zero=False, ax=ax1)
# Plot PACF to 8 lags (because only 7 days in a week) & ignore zero
plot_pacf(df_trans, lags=8, zero=False, ax=ax2)
# Zoom in on y axis to see points clearly
plt.ylim(-0.6, 0.6)
plt.show()
```

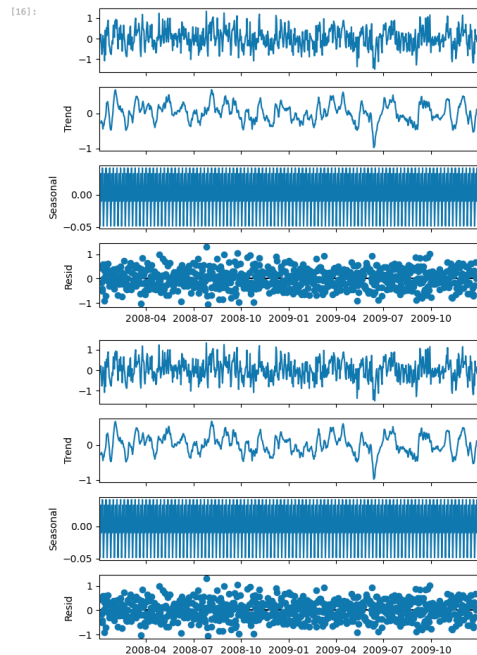
Spectral density:

```
# Plot spectral density
plt.psd(df['Revenue'])
plt.show()
```



Decomposed data plot:

```
# Plot decomposed data
decomposed_data.plot()
```



Confirm lack of trends in decomposition residuals:

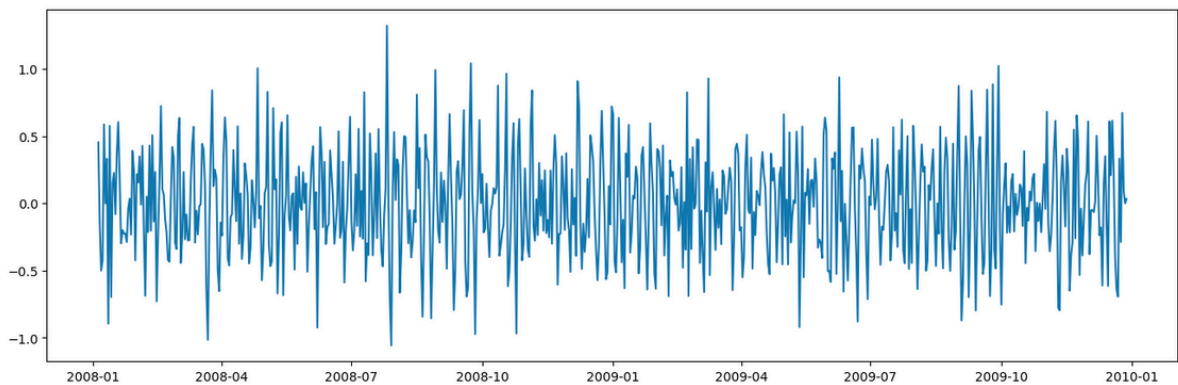
Show lack of trends in residuals of decomposition

plt.figure(figsize = [16,5])

Plot residual component of the data

plt.plot(decomposed_data.resid)

[17]: [



ARIMA model:

Create ARIMA model

model = ARIMA(train, order=(1, 0, 0), freq='D')

results = model.fit()

print(results.summary())

```

=====
SARIMAX Results
=====
Dep. Variable:      Revenue      No. Observations:      584
Model:              ARIMA(1, 0, 0)  Log Likelihood         -350.349
Date:              Wed, 20 Aug 2025  AIC                        706.698
Time:              10:37:15         BIC                     719.808
Sample:            01-02-2008       HQIC                     711.808
                  - 08-07-2009
Covariance Type:    opg
=====
              coef    std err          z      P>|z|      [0.025      0.975]
-----
const         0.0328     0.031        1.063     0.288     -0.028     0.093
ar.L1         0.4079     0.038       10.748     0.000     0.333     0.482
sigma2        0.1943     0.012       15.948     0.000     0.170     0.218
=====
Ljung-Box (L1) (Q):           0.10   Jarque-Bera (JB):           1.80
Prob(Q):                     0.75   Prob(JB):              0.41
Heteroskedasticity (H):       1.04   Skew:                  -0.05
Prob(H) (two-sided):         0.78   Kurtosis:              2.75
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

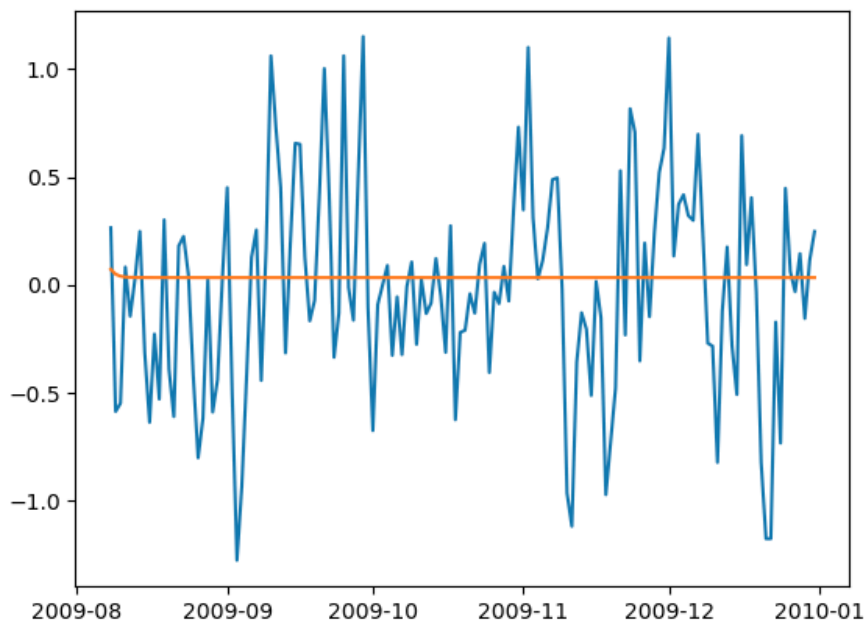
Forecast for ARIMA model:

Create forecast on ARIMA model

forecasted = results.get_prediction(start = 584, end = 729, dynamic = True)

plt.plot(test)

plt.plot(forecasted.predicted_mean)



Calculate Predicted mean values:

Print forecasted predicted mean

print(forecasted.predicted_mean)

```
2009-08-08    0.071071
2009-08-09    0.048405
2009-08-10    0.039159
2009-08-11    0.035388
2009-08-12    0.033850
...
2009-12-27    0.032791
2009-12-28    0.032791
2009-12-29    0.032791
2009-12-30    0.032791
2009-12-31    0.032791
Freq: D, Name: predicted_mean, Length: 146, dtype: float64
```

Forecast revenue differences:

Create dataframe to hold forecasted differences values

forecast_temp = pd.DataFrame(forecasted.predicted_mean)

Rename columns

forecast_temp.rename(columns={'predicted_mean' : 'Revenue'}, inplace=True)

Concat a copy of train and forecasted values

forecast_df = pd.concat([train.copy(), forecast_temp.copy()])

Invert the differences in daily revenue using cumsum

forecast_df = forecast_df.cumsum()

Print forecast_df

forecast_df

	Revenue
2008-01-02	-0.292356
2008-01-03	-0.327772
2008-01-04	-0.339987
2008-01-05	-0.124888
2008-01-06	-0.491590
...	...
2009-12-27	24.033683
2009-12-28	24.066474
2009-12-29	24.099265
2009-12-30	24.132056
2009-12-31	24.164846

730 rows x 1 columns

Confidence intervals:

```
# Calculate confidence intervals from forecasted data
```

```
confidence_intervals = forecasted.conf_int()
```

```
# Print confidence intervals
```

```
confidence_intervals
```

```
[22]:
```

	lower Revenue	upper Revenue
2009-08-08	-0.792856	0.934998
2009-08-09	-0.884621	0.981430
2009-08-10	-0.904871	0.983190
2009-08-11	-0.910461	0.981237
2009-08-12	-0.912301	0.980001
...	...	...
2009-12-27	-0.913421	0.979002
2009-12-28	-0.913421	0.979002
2009-12-29	-0.913421	0.979002
2009-12-30	-0.913421	0.979002
2009-12-31	-0.913421	0.979002

```
146 rows x 2 columns
```

```
# Create dataframe to match the confidence intervals dataframe
```

```
previous_row = pd.DataFrame({'lower Revenue': [19.312734], 'upper Revenue' : [19.312734],  
'Date' : ['2009-08-07']})
```

```
# Convert date string to datetime and then set as index
```

```
previous_row['Date'] = pd.to_datetime(previous_row['Date'])
```

```
previous_row.set_index('Date', inplace=True)
```

```
# Print
```

```
previous_row
```

	lower Revenue	upper Revenue
2009-08-07	19.312734	19.312734

```
# Concatenate prior row and the confidence intervals data
```

```
confidence_intervals = pd.concat([previous_row, confidence_intervals])
```

```
# Untransform the confidence intervals using cumsum()
```

```
confidence_intervals = confidence_intervals.cumsum()
```

```
# Omit first row
```

```
confidence_intervals = confidence_intervals.loc['2009-08-08' : '2009-12-31']
```

```
# Print confidence intervals
```

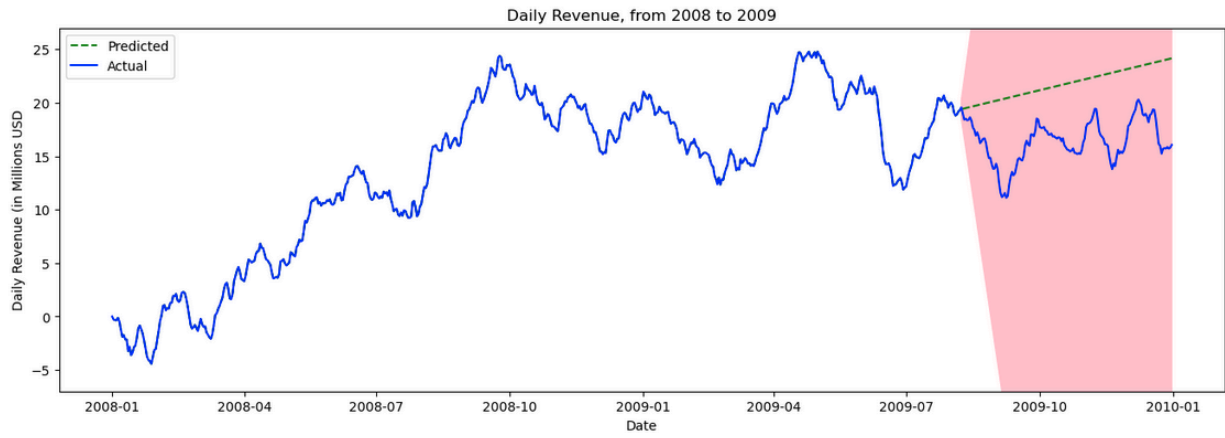
```
confidence_intervals
```

	lower Revenue	upper Revenue
2009-08-08	18.519878	20.247732
2009-08-09	17.635257	21.229163
2009-08-10	16.730386	22.212353
2009-08-11	15.819925	23.193590
2009-08-12	14.907624	24.173592
...	...	...
2009-12-27	-110.230261	158.297627
2009-12-28	-111.143681	159.276629
2009-12-29	-112.057102	160.255632
2009-12-30	-112.970522	161.234634
2009-12-31	-113.883943	162.213636

146 rows x 2 columns

Revenue Observed vs Predicted plot:

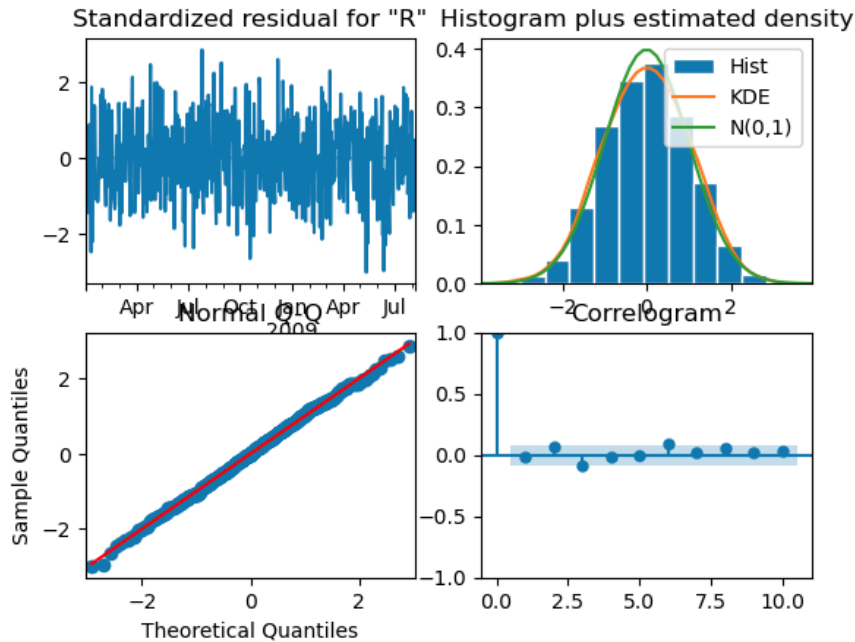
```
plt.figure(figsize = [16,5])
# Plot info
plt.title("Daily Revenue, from 2008 to 2009")
plt.xlabel("Date")
plt.ylabel("Daily Revenue (in Millions USD)")
# Plot the forecasted data
plt.plot(forecast_df, color = 'green', linestyle = 'dashed')
# Plot the original data
plt.plot(df, color = 'blue')
# Plot the confidence intervals
plt.fill_between(confidence_intervals.index, confidence_intervals['lower Revenue'],
confidence_intervals['upper Revenue'], color = 'pink')
# Keep the y-axis zoomed in
plt.ylim(-7, 27)
# Legend for plot
plt.legend(['Predicted', 'Actual'])
plt.show()
```



Plot diagnostics:

```
plt.figure(figsize = [16,16])
```

```
results.plot_diagnostics()
```



Calculate RMSE:

```
# Calculate root mean squared error of forecasted data vs the observed data
rmse = root_mean_squared_error(df.loc['2009-08-08' : '2009-12-31'],
forecast_df.Revenue.loc['2009-08-08' : '2009-12-31'])
print(f"The root mean squared error of this forecasting model is {round(rmse, 5)}")

The root mean squared error of this forecasting model is 5.7584
```

Section D5. Code

Code attached in “d213-task1.ipynb”

Part V: Data Summary and Implications

Section E1. Results

An ARIMA(1,0,0) model was chosen for this analysis after reviewing the autocorrelation and partial autocorrelation plots which showed that an autoregressive model of order one would best fit the data.

The forecast was made for the final 20% of the dataset (from August to December 2009). This length was chosen because it allowed for a comparison between predicted values and actual

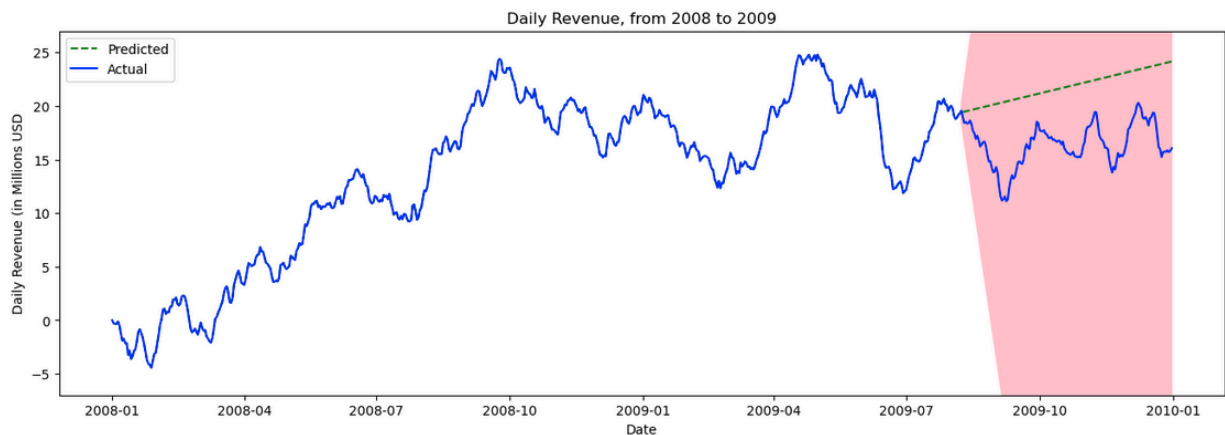
observed data. Forecasting into the future without observed data would not allow the model's accuracy to be evaluated effectively.

Prediction intervals were also created for the forecast. At the beginning of the test period, the intervals were relatively narrow (from about \$18.5 million to \$20.2 million), meaning the model was more confident. However, as time went on, the gap widened significantly, reaching very high ranges by the end of the year (from less than -\$110 million to over \$160 million). This shows that the model became less reliable over longer periods of time.

The model was evaluated using residual diagnostics and RMSE. The diagnostic plots showed that the errors were mostly random and normally distributed, however there were some signs of correlation at certain lags (around lag 3 and lag 6). RMSE was calculated at 5.7584. A lower RMSE means a better fit for the model. There is no threshold for what would be considered a “good” or “bad” RMSE, but it could be used to compare to other models.

Section E2. Annotated Visualization

Below is the annotated forecast vs actual revenue plot. The results are a forecast that has an increase in daily revenues, however the actual pattern in the data is not reflected. In reality, revenues declined during the forecast period, which the model was unable to capture. The model oversimplifies the data by projecting only a gradual increase, which shows that it is not capable of accurately reflecting the real behavior of the hospital's revenues.



Section E3. Recommendation

Based on the results of this analysis, the recommendations section is to treat the ARIMA(1,0,0) model as a stepping stone rather than a reliable forecasting tool. The model shows that daily revenue could be forecasted in theory, but lacks in accuracy especially over a long period of time. Additionally, hospitals should not rely on this forecast for financial planning since it doesn't take potential downturns in consideration. Instead, in order to get more accurate numbers, the forecasts should be used alongside actual revenues and updated regularly as new data comes in.

Part VI: Reporting

Section F. Reporting

Jupyter notebook (in pdf format and ipynb) is attached.

Section G. Sources for Third-Party Code

Converting day count to date time. (2020, April 23). Stack Overflow.

<https://stackoverflow.com/questions/61389654/converting-day-count-to-date-time>

Section H. Sources

Fulton, J. (n.d.). *Intro to ACF and PACF*. DataCamp. Retrieved August 20, 2025, from

<https://campus.datacamp.com/courses/arima-models-in-python/the-best-of-the-best-models?learningMode=course&ex=1>

Jain, S. (2023, October 20). *Time Series Decomposition Techniques*. GeeksforGeeks. Retrieved August 20, 2025, from

<https://www.geeksforgeeks.org/python/time-series-decomposition-techniques/>

Jain, S. (2024, February 28). *Power Spectral Density*. GeeksforGeeks. Retrieved August 20, 2025, from

<https://www.geeksforgeeks.org/electronics-engineering/power-spectral-density/>

Kuo, A. (2024, June 12). *What is Time Series Analysis?* Julius AI. Retrieved August 20, 2025, from <https://julius.ai/articles/what-is-time-series-analysis>

NIST. (n.d.). 6.4.4.2. *Stationarity*. Information Technology Laboratory. Retrieved August 20, 2025, from <https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc442.htm>

Prabhakaran, S. (2019, November 2). *Augmented Dickey Fuller Test (ADF Test) – Must Read Guide*. Machine Learning Plus. Retrieved August 20, 2025, from <https://www.machinelearningplus.com/time-series/augmented-dickey-fuller-test/>